

VEHICULAR AKASHA

A Multi-Vehicle Telemetry System

ECE 499 Capstone Design Proposal

Student Name: Jesutobi Onigbogi

Student ID: 2667737

Supervisor: Mohammed Umair

Date: 20 March 2026

Course: ECE 498 - Capstone Design

Table of Contents

Table of Contents	2
Section I: Project Definition	3
I.A. Background and Technology Survey	3
I.B. Needs Statement	4
I.C. Objectives Statement	5
Section II: Project Design Requirements	6
II.A. Specifications	6
II.B. Constraints	6
II.C. Standards and Codes	7
Section III: System Design & Functional Decomposition	10
III.A. Level 0 Block Diagram	10
III.B. Level 1 Block Diagram	10
III.C. Module Definitions	11
Section IV: Design Alternatives	18
IV.A. Microcontroller Alternatives	18
IV.B. Wireless Communication Alternatives	19
IV.C. CVT Belt Temperature Sensor	19
Section V: Subsystem Design	21
V.A. OBD-II [CAN Interface Subsystem]	21
V.B. LoRa Communication Subsystem	21
V.C. Serial Data & SD Card Storage	22
Table 5, above shows hours of constant data reading one could expect of the system, of course since it makes use of expandable storage a user can upgrade the current 32 GB storage to 64 GB or 128GB and get more runtime.	24
Data Transmission and Storage Calculations	24
V.D. ILI9341 Dashboard Display	25
V.E. Microcontroller Processing	26
V.G. MATLAB Receiver code	28
Section VI: Final Design	30
VI.A. Final Schematics	30
VI.B. Bill of Materials	31
VI.C. Power Consumption Analysis	32
VI.D. Data Format Specification	32
Section VIII: Final Results and Design Evaluation	34
VII.A. LoRa Communication Test	34
VII.B. MPU 6050 Acceleration and Gyro Test	36
VII.C. Temperature Sensor Test	37
VII.D. GPS Test	38
VII.E. Micro SD Card Reader Test	40

VII.G. UI + ILI9341 Test	43
VII.H. Battery System Test	44
Section VIII: Ethical Considerations	46
Section IX: Project Schedule	47
References	49
Appendices	50
Appendix A: Competitive Benchmarks	50
Appendix B: ESP32 Nano Pin Layout	50
Appendix C: PCB Layout	51
Appendix C: OBDII Data Survey Statistics Preferences	52

Section I: Project Definition

I.A. Background and Technology Survey

Modern vehicles generate extensive operational data through onboard diagnostics (OBD-II) systems, yet accessing and consolidating this information across multiple vehicles remains challenging for average owners. The average American household owns nearly two vehicles, with many owning three or more [1]. Fleet operators, rental companies, and automotive enthusiasts face even greater challenges monitoring vehicle health across their assets. The global vehicle telematics market is projected to grow significantly, demonstrating strong demand for comprehensive monitoring solutions [2].

Current solutions fall into distinct categories with notable limitations. Commercial telematics systems provide comprehensive fleet management but require substantial subscription costs of \$20–50 per vehicle monthly, targeting commercial operations rather than individual consumers [3]. Consumer-grade OBD-II dongles offer basic diagnostics but typically lack historical logging, multi-vehicle views, and remote monitoring capabilities.

The OBD-II standard, mandated for all U.S. vehicles since 1996, provides standardized diagnostic access through the SAE J1979 protocol [4]. Modern embedded systems, particularly microcontrollers with integrated wireless capabilities such as the ESP32 family which make sophisticated data collection feasible at accessible price points. The emergence of low-power wide-area network (LPWAN) technologies, particularly LoRa, provides long-range wireless communication ideal for off-road and competition environments where cellular coverage is unreliable.



Figure 1: Baja Kart in Competition

This project, Vehicular Akasha, addresses key market gaps through affordable comprehensive monitoring with a one-time hardware cost under \$75, multi-vehicle data consolidation, independent data verification through GPS and accelerometer sensors, a customizable on-board display, and an open-platform design. The system is also designed for the University Baja Racing program, enabling telemetry on non-standard vehicles that lack native OBD-II buses.

Sustainability considerations include extended vehicle lifespan through proactive maintenance, emission tracking for environmental awareness, reusable and modular hardware components, and an open-source approach that prevents planned obsolescence.

I.B. Needs Statement

Vehicle owners managing multiple vehicles, small fleet operators, and automotive enthusiasts, need an affordable and comprehensive solution to monitor and analyze vehicle performance across their fleet in one centralized location. Current solutions are either prohibitively expensive for individuals, limited in functionality, or restricted to specific manufacturers.

According to AAA, average annual vehicle maintenance costs exceed \$1,000 per vehicle [5], with many resulting from delayed issue detection. For the Baja Racing team specifically, there is no affordable off-the-shelf solution for acquiring telemetry data from custom-built karts that lack OBD-II buses. Existing racing telemetry systems cost \$500–\$2,000+ and are designed for high-end motorsport.

Stakeholders include individual owners with multiple vehicles, small business fleet managers, automotive enthusiasts, car rental operators, and university racing teams operating custom vehicles.

I.C. Objectives Statement

Develop Vehicular Akasha, an affordable vehicle telemetry device providing comprehensive real-time monitoring and historical analysis across multiple vehicles through a cloud dashboard, PC serial interface, and customizable onboard ILI9341 display.

The system monitors vehicle parameters from two independent sources: OBD-II (engine RPM, coolant temperature, vehicle speed, and 15+ parameters) and independent sensors (GPS position/speed, 3-axis IMU, CVT temperature via MLX90614, and structural strain via BF350 gauge). This dual-source approach eliminates OBD-II reporting bias and allows continued data collection during CAN bus failures.

For non-standard vehicles such as Baja karts, the system translates all sensor readings into CAN frames using the TJA1051 transceiver, creating a standards-compliant data bus on a platform that otherwise has none.

Section II: Project Design Requirements

II.A. Specifications

The following table maps marketing requirements to quantitative engineering specifications:

Marketing Requirement	Engineering Specification	Justification
Affordable solution	Hardware cost < \$75	Undercuts \$240–600/yr commercial solutions
Wireless Communication	190 Bps; <5% loss; 5 km LoRa	Reliable data over Baja terrain
Multi-vehicle support	10+ vehicles per account	Household and small business needs
Real-time monitoring	Data ≥ 1 Hz; Display ≥ 30 FPS	Smooth visual feedback to driver
Easy installation	< 5 minutes	Plug-and-play OBD-II connector
Comprehensive data	15+ OBD-II params; GPS; 3-axis accel	Independent verification
Data verification	GPS ± 2 mph; alert on >10% difference	Detect faulty sensors
Battery-safe	Quiescent < 80 mA	Prevent drain during parking
Reliable operation	-20°C to 70°C	Automotive environment extremes
Storage	32 GB SD at ≥ 100 KB/s	Long-term drive records

Table 1: Marketing-to-Engineering Requirements Matrix

II.B. Constraints

Economics / Cost: Total hardware must remain under \$75. Target retail \$99–129 provides margins while remaining accessible to individuals and student teams.

Health and Safety: Strictly read-only OBD-II, no ECU write commands. Reverse-polarity diodes, overcurrent fuses, ESD protection on CAN lines. Watchdog timers and fail-safe firmware.

Environmental: Operating range -20°C to 70°C , 90% humidity, vibration per IEC 60068-2-6 [8]. RoHS-compliant materials. For Baja use: heavy mud, dust, water splashes, sustained off-road vibration.

Baja SAE Team Dynamics: The Baja team comprises data acquisition, drivetrain, hubs/brakes, suspension, and cabin subsystems. Component sizing and power consumption cannot be decided in isolation—the entire vehicle’s power budget and physical space must be shared. Cross-team coordination overhead constrains development timelines and testing access.

Baja SAE Rules and Regulations: The fuel tank must remain unmodified from factory form, so no fuel level sensor is included [11]. On-vehicle battery charging during operation is prohibited, directly driving the 10-hour minimum battery life requirement [11].

Legal / Regulatory: FCC Part 15/15.247 for 915 MHz ISM band [7]. GDPR/CCPA privacy compliance. Users maintain explicit data control.

Manufacturability: Standard 2-layer PCB with common SMD/through-hole packages. Needed to secure components to housing for continued data logging.

Usability: 5-minute install. OBD-II plug-and-play for standard vehicles; labeled harness for karts. SquareLine Studio dashboard. Plain-language error messages.

II.C. Standards and Codes

SAE J1979: OBD-II Diagnostic Test Modes establishes the standardized language that vehicles and diagnostic tools use to communicate. It defines the Parameter IDs (PIDs) that the system queries to retrieve engine RPM, vehicle speed, coolant temperature, and dozens of other metrics. Because this standard has been mandated for every passenger vehicle sold in

the United States since 1996, Vehicular Akasha can interface with virtually any car on the road without manufacturer-specific adapters or proprietary protocols. The standard specifies nine diagnostic modes covering real-time data streaming, freeze-frame retrieval, and diagnostic trouble code reading, giving the system a broad and well-documented foundation to build on [4].

ISO 15765-4 and ISO 11898-2: CAN Bus Protocol and Physical Layer govern how data actually moves between the vehicle's electronic control units. CAN (Controller Area Network) is the backbone communication standard in modern automobiles — it allows dozens of modules like the engine controller, transmission, ABS, and instrument cluster to share data over a single twisted-pair wire bus. ISO 11898 defines how messages are formatted, prioritized through arbitration, and protected from errors using CRC checksums. The physical layer specification (ISO 11898-2) sets the electrical characteristics that the TJA1051T/3 transceivers must meet: differential signaling at 500 kbps with 120 Ω bus termination. These standards ensure that the system communicates reliably with any CAN-equipped vehicle and that the CAN frames it generates for non-standard vehicles are fully compliant with off-the-shelf analysis tools [6].

SAE J2284: CAN Signal Quality further specifies the voltage levels, rise times, and common-mode rejection requirements that CAN transceivers must meet in passenger vehicle environments. This standard ensures that the system's signals remain clean and readable even in the electrically noisy conditions under a vehicle's hood, where ignition coils, alternators, and electric motors generate significant electromagnetic interference.

FCC Part 15: Radio Frequency Emissions regulates all electronic devices that emit radio energy in the United States. The LoRa RYLR998 module operates at 915 MHz within the unlicensed ISM (Industrial, Scientific, and Medical) band, which means it can transmit without requiring a special license or permit. Under Part 15.247, spread-spectrum devices in this band are allowed a maximum conducted output power of 1 watt (30 dBm), though the RYLR998

typically operates well below this ceiling. Compliance ensures the system can be legally deployed at any competition venue or on public roads without regulatory concerns [7].

Baja SAE Competition Rules impose additional constraints specific to the racing environment. All electrical systems must be securely mounted and protected from water, mud, and debris exposure during competition events. Wiring must be routed away from hot or moving components, and all connections must be mechanically secure against the sustained vibration of off-road driving. The rules also prohibit any modification to the stock fuel tank and forbid on-vehicle battery charging during operation, both of which directly influenced the system's sensor selection and power budget as discussed in Section II.B [11].

Section III: System Design & Functional Decomposition

III.A. Level 0 Block Diagram

The Level 0 diagram shows the system as a single functional block with defined inputs and outputs.

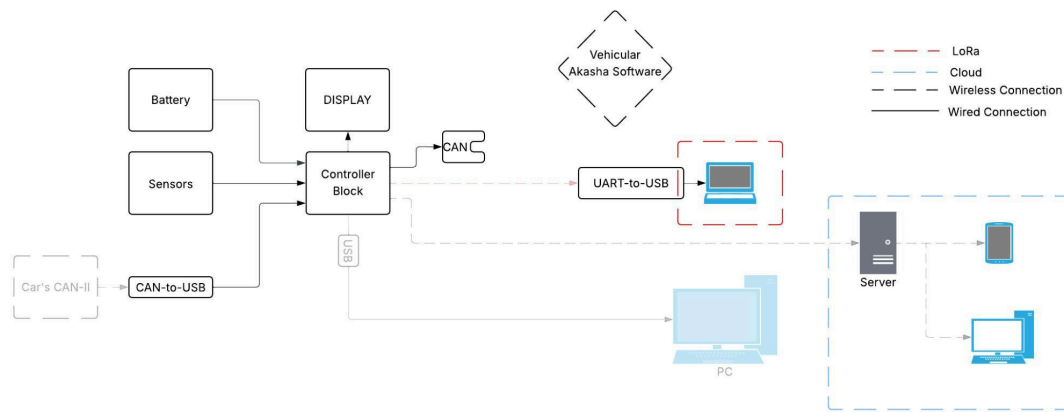


Figure 2: Level 0 System Block Diagram - LoRa (red dashed), Cloud (blue dashed), Wireless (black dashed), Wired (solid).

Inputs: Vehicle CAN Bus, GPS Signals, Vehicle Motion, 12V Power, WiFi Credentials, User Commands, CVT Temperature, Structural Strain.

Outputs: Visual Display (ILI9341), PC Serial Data, LoRa Packets, SD Card Logs (.CSV), CAN Frames, Status LEDs.

III.B. Level 1 Block Diagram

The system decomposes into modules centered on the ESP32. Sensors feed the controller, which routes CSV data to four outputs: display (SPI), SD card (SPI shared bus), LoRa (UART), and USB serial. CAN transceivers provide OBD-II input and standards-compliant output for non-standard vehicles.

III.C. Module Definitions

Each module is presented with a general overview and the key specifications that drove its selection.

Module 1: OBD-II / CAN Bus Interface

The TJA1051T/3 CAN transceiver (CAN_INPUT) converts differential CAN signals from OBD-II pins 6/14 to logic levels for the ESP32's TWAI controller, eliminating the need for an external MCP2515. A second TJA1051T/3 (CAN_OUTPUT) transmits sensor-derived CAN frames for non-standard vehicles, creating a standards-compliant bus on karts. The ESP32 sends Mode 01 PID requests at 500 kbps with 120Ω termination.

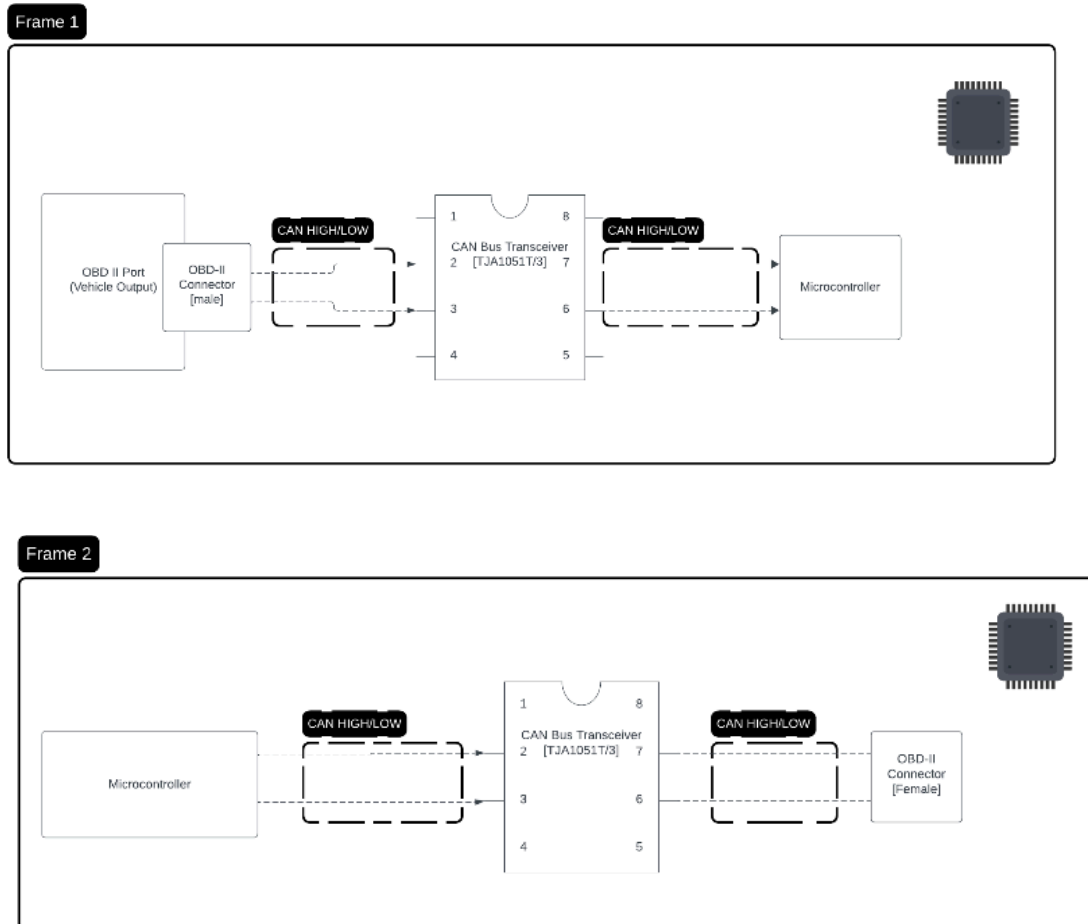


Figure 3. CAN bus interface showing Frame 1 (Vehicle read path) and Frame 2 (Path for non-standard vehicles)]

Module 2: Microcontroller - Arduino Nano ESP32

Microcontroller acting as the central brain concurrently supporting the CAN communication; the display rendering, LoRa transmission, serial communication and SD writes. Natively supports SPI, UART, I²C, and TWAI (CAN) as necessary for the project.

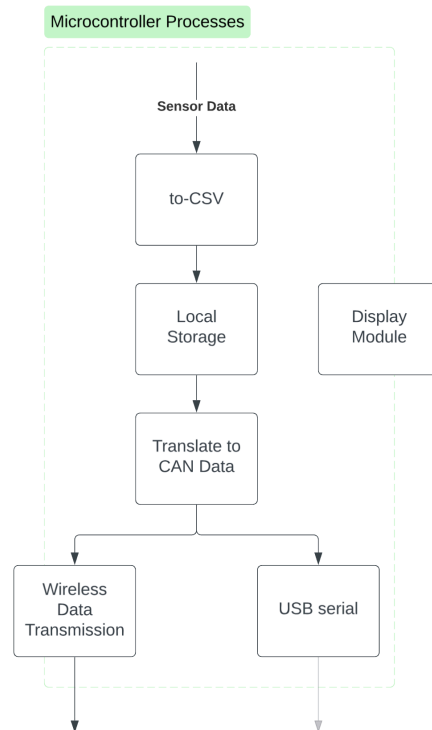


Figure 4. Microcontroller Processes

Above is an overview of the microcontroller general processes and how it interacts with other subsystems to deliver the proposed solution.

Module 3: USB Serial Interface

PC-side SerialRW.py receives CSV frames and logs to data.csv. GraphRPM.py renders real-time animated matplotlib plots with auto-scrolling x-axis. Also supports firmware flashing and Arduino IDE serial debugging.

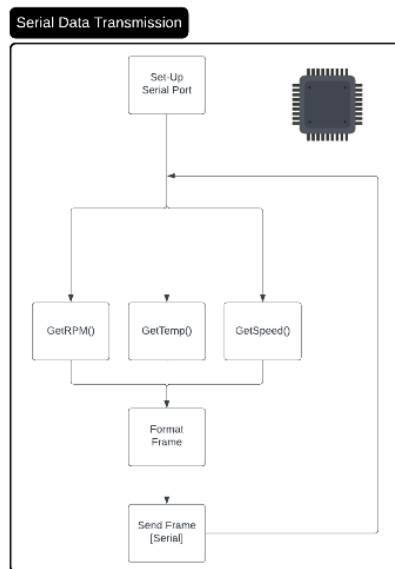


Figure 5 - Serial data flow diagram - ESP32 to PC, SerialRW.py and GraphRPM.py pipeline

Module 4: GPS NEO-M8M

Satellite-derived position, velocity, and timing independent of vehicle sensors. GPS give more accurate speed data over MPU6050

Module 5: MPU6050

6-axis inertial measurement unit: 3-axis accelerometer ($\pm 16g$) and gyroscope ($\pm 2000^\circ/s$). Captures hard braking, cornering g-loads, terrain impacts, and rollovers. I²C at address 0x68, sampled at 100 Hz and downsampled to 1 Hz for LoRa while retaining high-rate data on SD card. 3.9 mA active current.

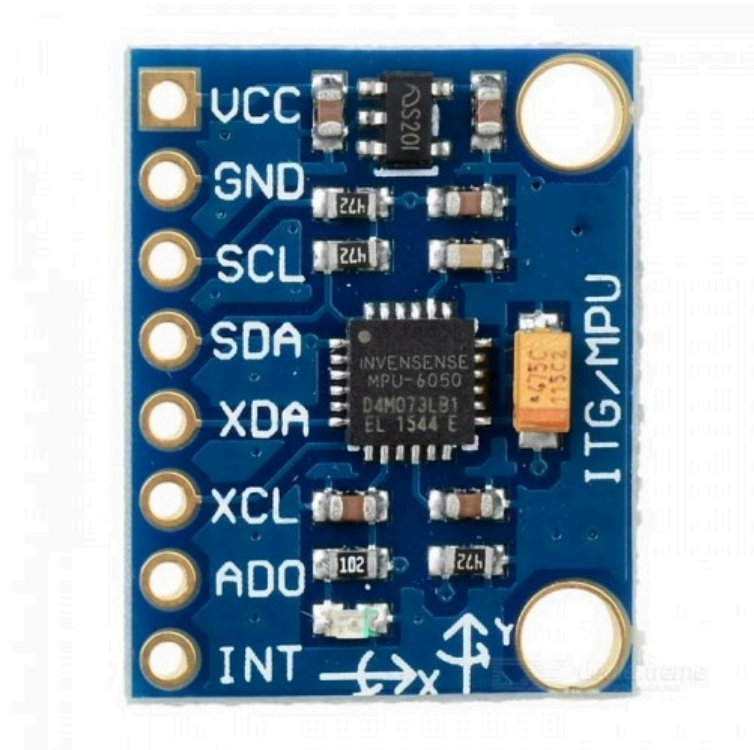


Figure 6. MPU6050 breakout board

Module 6: IR Temperature MLX90614

Non-contact infrared thermometer for CVT belt monitoring. Provides both IR object temperature (-70°C to 380°C) and ambient die temperature in a single I²C read at address 0x5A. Factory-calibrated $\pm 0.5^{\circ}\text{C}$ accuracy. 1.5 mA draw produces negligible self-heating inside the enclosed CVT casing.



Figure 7. MLX90614 sensor

Module 7: Strain Gauge BF350-3AA

Foil-type resistive gauge bonded to the kart's tubular steel frame. Wheatstone bridge configuration with instrumentation amplifier conditioning for ESP32's 12-bit ADC. Detects frame fatigue and structural overload in real time. Purely passive at <1 mA. 350Ω nominal resistance, gauge factor ~ 2.0 , output in micro-strain (m/m).

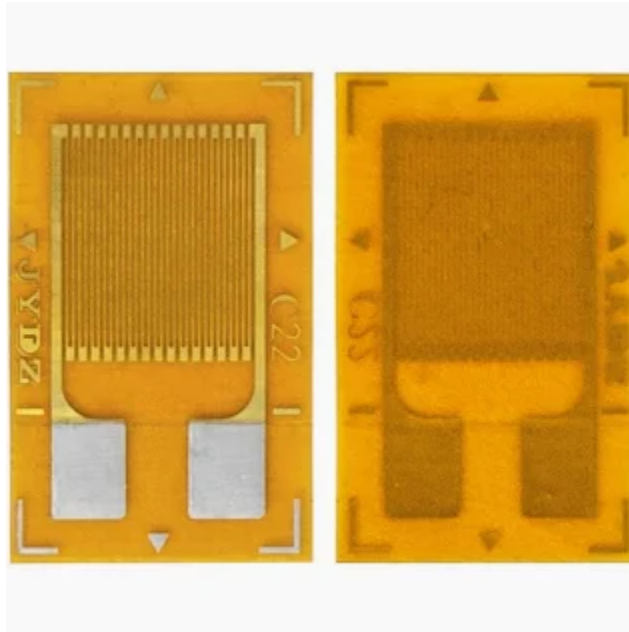


Figure 8. Strain gauge

Module 8: Display with ILI9341 (2.8" TFT)

320×240 pixel color touchscreen via SPI. Custom LVGL dashboard designed in SquareLine Studio with circular speed gauge, status bar, and screen switching. ≥ 30 FPS via DMA-accelerated SPI. PWM backlight control. Resistive touch enables mode changes without physical buttons that could fail from vibration.

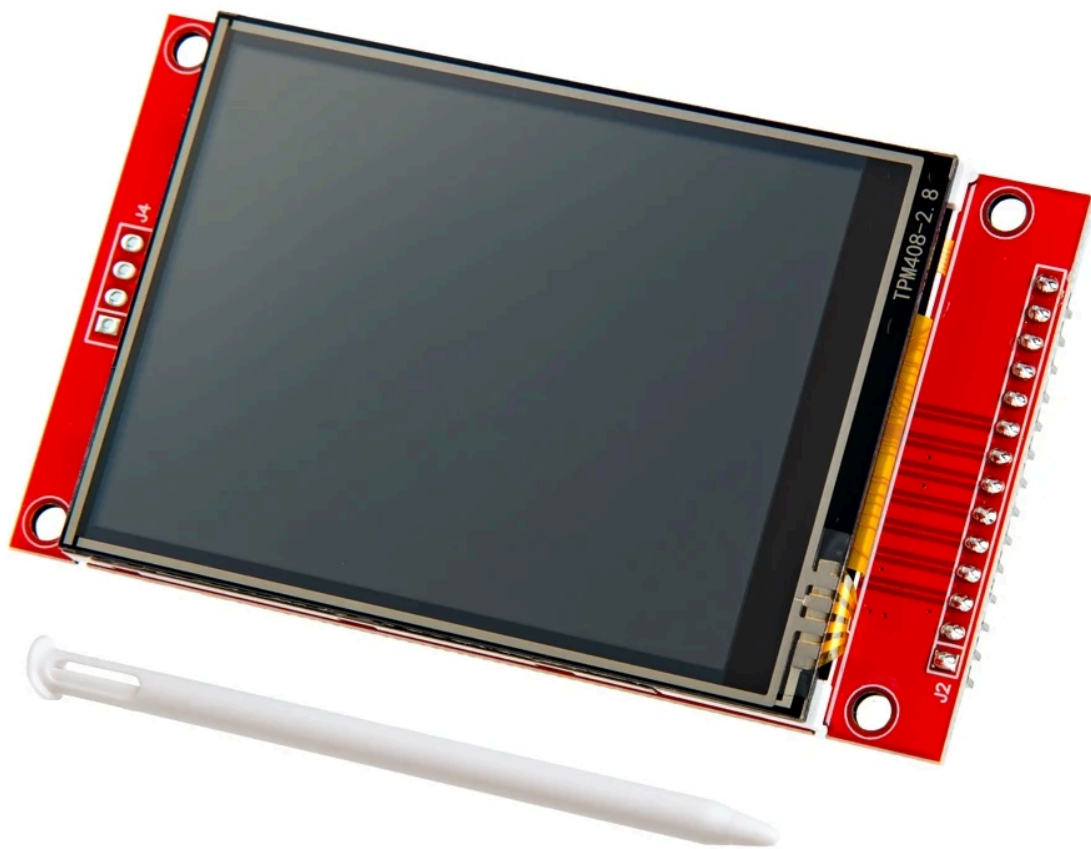


Figure 9. ILI9341 for dashboard UI

Module 9: SD Card Storage

Primary archival medium—captures every data point even when LoRa has packet loss or USB is disconnected. MicroSD via SPI (shared bus with display, independent CS). Writes CSV at

200–500 KB/s, far exceeding the 120 B/s requirement. .CSV files directly importable into Excel, MATLAB, or Python allowing for interfacing and expansion onto those three fronts.

Module 11: LoRa RYLR998

915 MHz ISM band transceiver for 5–10 km pit-to-vehicle communication. UART-connected to ESP32; configured via AT commands. 37.5 kbps max capacity vs. 320 bps actual usage (0.85% utilization). Matched receiver at pit stop connects via CP2102 USB-UART converter. See datasheets for AT commands.



Figure 10. RYLR998 transmitter (on-vehicle) and receiver (pit-stop with CP2102)

Module 12: PC Application

MATLAB live data graphing, using COM serial, requiring the installation of CP2102 drivers to function (some PCs come with it while others might have to install it).

Section IV: Design Alternatives

IV.A. Microcontroller Alternatives

Feature	Arduino Nano ESP32	Teensy 4.1
CPU	Dual-core Xtensa LX6, 240 MHz	ARM Cortex-M7, 600 MHz
Wireless	Built-in WiFi + Bluetooth	None (external module)
CAN	Built-in TWAI	FlexCAN (native)
Community	Very large ESP32 ecosystem	Smaller, audio/DSP focused
Cost	~\$10–15	~\$30–35
Power	40–50 mA	~100 mA

Table 2: Microcontroller Comparison

Selection: Arduino Nano ESP32. It has native WiFi and Bluetooth, with CAN (referred to as TWAI in documentation), lower cost, makes use of less power especially on lengthy drives this proves to be especially useful allowing for longer rides without battery depletion. The 240 MHz sufficient as subsystems will bottleneck system clocking anyways it proves not to be a dealbreaker. Moreover and most importantly it has libraries I am familiar with and a large community to leverage for assistance with Teensy lacks. Generally whatever ESP32 lacks is made up for with ease of use and IoT capabilities, in addition it can run python, while this is a feature not currently utilized in the design it opens up avenues for future iterations. Can easily handle 30+ FPS display, and concurrent comms.

IV.B. Wireless Communication Alternatives

Technology	Pros	Cons
Wi-Fi	High speed, easy setup	Short range (<100 m)

Cellular (4G/5G)	Wide urban coverage	SIM + subscription; unreliable remote
LoRa (915 MHz)	5–10 km; reliable off-road; no fees	Low rate (190 Bps)

Table 3: Wireless Communication Alternatives

Selection: LoRa (RYLR998). The reasoning for this is specific to the BAJA competition terrain which sometimes can support cellular but often doesn't, as such LoRa is the most suitable. Low data and smaller data amounts is fine since only telemetry data is being sent across. Specifically the 320 bps telemetry well within 190 Bps which is beyond excellent for its purpose of use. Unlike Cellular do not have to deal with recurring costs for SIM. To surmise, for the geographical versatility and long range it makes for the clear winner and most suitable option

IV.C. CVT Belt Temperature Sensor

A contactless sensor is required because the CVT belt is in continuous motion. An infrared sensor aimed at the belt surface provides more accurate readings than ambient measurement alone. Ideally, a sensor providing both IR and ambient readings enables dual-measurement for calibration and casing condition awareness. Expected range: 5°C to 150°C. Must use I²C/SPI, cost <\$30, and draw minimal current to avoid self-heating artifacts.

Feature	MLX90614	AMG8833	TMP006
IR Range	−70 to 380°C	0 to 80°C	−40 to 125°C
Accuracy	±0.5°C	±2.5°C	±1.0°C
Interface	I ² C	I ² C	I ² C
Current	1.5 mA	4.5 mA	0.3 mA
Cost	\$8	\$25	\$8

Table 11: IR Temperature Sensor Comparison

Selection: MLX90614. Three infrared sensors were considered for monitoring the CVT belt temperature. Firstly, the AMG8833 is essentially a miniature thermal camera whilst a good

choice for a lot of projects and very high end, it is complete overkill for pointing at a single belt, and at \$25–35 it eats a significant chunk of the project's hardware budget. The TMP006 is the cheapest to run power-wise, but its measurement ceiling tops out right around the temperatures the belt actually reaches; it can be difficult to capture temperatures if things get hotter than expected during a long endurance run. The MLX90614 can comfortably handle higher temps as it reads well beyond what the belt will ever reach, it is accurate enough to catch onto early overheating and at around \$8 making it the most affordable option of the three thus leaving more budget available for other parts of the system. All three talk over I²C so compatibility was not a differentiator, it really came down to the MLX90614 giving the most useful range at the best price without any unnecessary extras.

Section V: Subsystem Design

V.A. OBD-II [CAN Interface Subsystem]

Two TJA1051T/3 transceivers: CAN_INPUT reads from OBD-II pins 6/14 (CAN-H/L) and converts differential signals for the ESP32 TWAI controller. CAN_OUTPUT transmits sensor-derived CAN frames for non-standard vehicles.



Figure 11: OBD-II Connector with Color-Coded Wire Reference

V.B. LoRa Communication Subsystem

Employs a transmitter RYLR998 on-vehicle via UART and a receiver combo of a RYLR998 + CP2102 USB-UART at pit stop. Below is a table of setting parameters and commands for the transmitter-reciever pair.

LoRa Parameter Settings		
Parameter	Transmitter	Receiver
AT+MODE=	0	0
AT+ADDRESS=	1	2
AT+NETWORKID=	5	5
AT+BAND=	915MHz	915MHz
AT+PARAMETER=	9,7,1,12 <default>	

Table System Lora AT command settings

V.C. Serial Data & SD Card Storage

The serial and SD card subsystems work together to ensure that telemetry data is both accessible in real time and preserved for later analysis. On the serial side, the ESP32 continuously polls each sensor, assembles the readings into a comma-separated string, and pushes it out over USB at 115,200 baud. On the PC end, a Python script called SerialRW.py listens on the COM port, catches each incoming frame, and writes it straight to a CSV file. A companion script, GraphRPM.py, reads that same CSV and draws a live-updating chart so the pit crew can watch vehicle performance unfold in real time without waiting until the car comes back in. For long-term storage, a microSD card module handles local logging directly on the vehicle. It shares the SPI bus with the ILI9341 display but uses its own independent chip-select line, so the two devices take turns without stepping on each other.

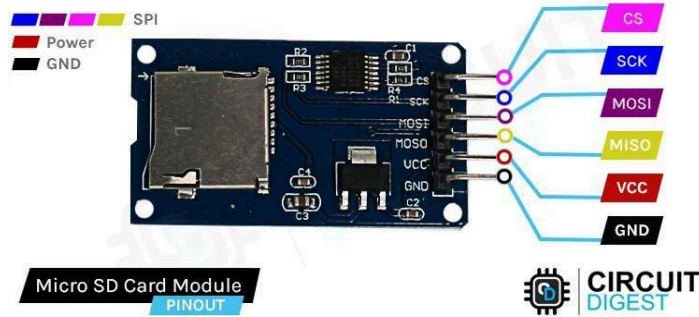


Figure 12: SD Card Module SPI Wiring

Table 5. Storage Capability Calculations

Table 5, above shows hours of constant data reading one could expect of the system, of course since it makes use of expandable storage a user can upgrade the current 32 GB storage to 64 GB or 128GB and get more runtime.

Data Transmission and Storage Calculations

Field	Pac. #	Accel X, Y, Z (in g)	Gyro X, Y, Z	CVT temp (F/C)	Strain (N)	Speed (Mph)	latitude	longitude	No of Satellite(s)	Separation(l)	Total
Bytes	5	18	21	5	7	5	11	11	2	14	99 Bytes
Hardware Rate (Hz)	-	1000	800 0	100k	10/80	1	1	1	1	-	
Total (Bps)	-	18k	168k	500k	70/56 0	5k	11	11	2	-	841k Bps
Adjusted Rate [DATA SAMPLING RATE] (Hz)	5	5	5	5	5	5	5*	5*	5*	5	
[SD CARD] Total (Bps)	25	25	105	25	35	25	55	55	10	70	375 Bps

Adjusted Rate [LoRa]											
Rate (Hz)	1	1	1	1	1	1	1	1	1	1	
[LoRa] Total (Bps)	5	18	21	5	7	5	11	11	2	14	99 Bps

The highlighted in the figure above shows the Sampling Data rate, SD Card data system write rate and the wireless LoRa rate. Note the (*) on the Speed, Latitude and Satellite are derived from the GPS which updates at a max of 1 Hz which can not be upscaled, hard bottlenecking the system rate, having to combine the need more frequent sampling data whilst not compromising for data validity, the design has been compromised to account for both, the system takes data at 1Hz and carries over GPS data (Speed, Latitude, Longitude and Satellite) over 5 reading in 1 second. The Total Bps is the maximum data rate needed for maximum hardware.

V.D. ILI9341 Dashboard Display

The primary purpose of the 2.8-inch 320×240 TFT is to render a custom interface made with LVGL UI from SquareLine Studio to show the following statistics to the driver as it will be on-board the vehicle. An realistic expectation of ≥ 30 FPS is the bare minimum nowadays so will hold the project to this standard.

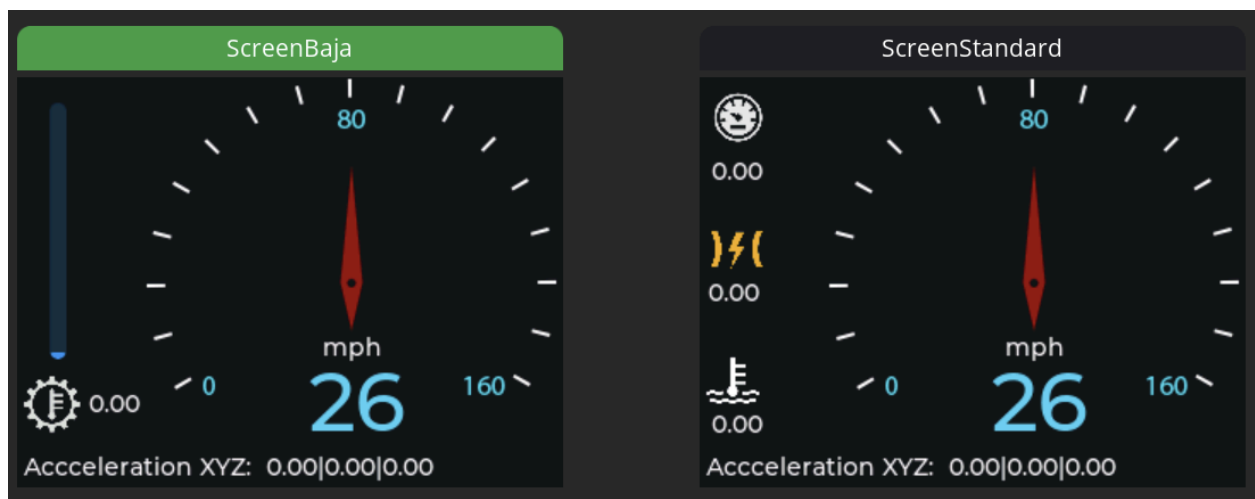


Figure 13: Dashboard UI (left is UI without OBDII port connected, right is with OBDII Connection)



Figure 14: ILI9341 Prototype on Breadboard with ESP32 and CAN Transceivers

When OBDII port is unconnected, shows screen on left in Figure 14, having a bar visualizer for temperature capping at 500 Fahrenheit for and a moving pointer for GPS speed. If OBDII port is connected reads in car telemetry data for RPM, throttle position, engine coolant temperature and vehicle speed (on pointer).

V.E. Microcontroller Processing

The process outlined above is structured as a series of sequential and concurrent stages designed to move data from raw acquisition through to multiple output channels. It begins with sensors collecting data, where each sensor is sampled at its native rate to preserve the full fidelity of the incoming data this means high-frequency sensors like accelerometers or gyroscopes are read far more often than slower peripherals such as temperature or pressure sensors, ensuring no critical readings are missed. Once collected, the raw data is consolidated and formatted into CSV at a uniform standard that normalizes the varying sample rates into a unified time-series record. This formatted data is then committed to the SD card for persistent, on-device storage, providing a reliable local log that survives power cycles and connectivity interruptions. In parallel, the display is continuously refreshed via the LVGL graphics library at a minimum of 30 frames per second, delivering a smooth and responsive visual interface for real-time monitoring of key parameters. For vehicles that do not conform to standard OBD-II or

J1939 protocols, a CAN translation layer interprets and remaps non-standard message frames into a consistent internal format, allowing the system to interface with a wide variety of vehicle architectures without requiring firmware changes for each new platform. Telemetry is then transmitted wirelessly over LoRa using AT+SEND commands. Finally, a USB serial that mirrors the data stream to a connected computer, serving as both a debugging tool during development and a secondary output channel for external logging or real-time analysis software.

V.F. Battery System

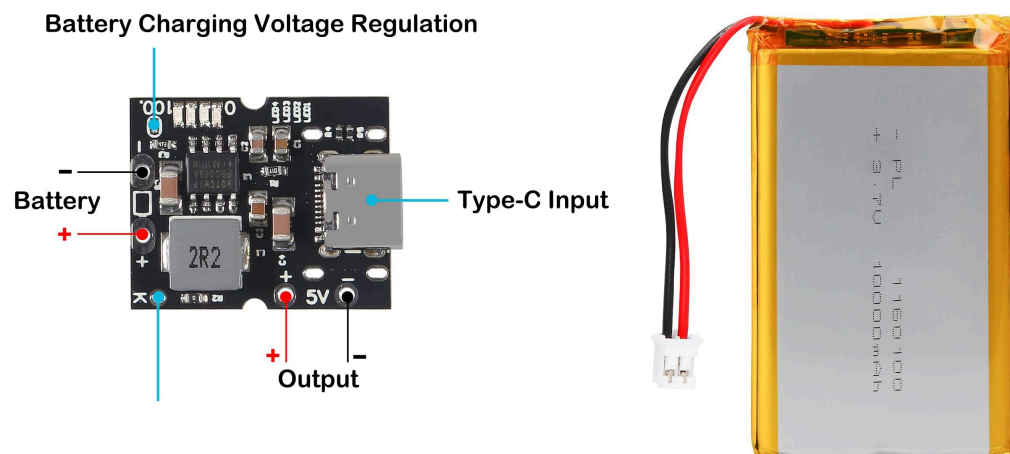


Figure 15. DWEll 10PCS Type-C USB 5V 2A Boost Converter Step-Up with 3.7 10,000mAh Battery

The system is powered by a single 3.7V 10,000 mAh lithium-polymer cell connected to a Type-C USB 5V 2A boost converter module. The boost converter steps the battery's nominal 3.7V output up to a regulated 5V rail, which supplies the ESP32 Nano and all peripheral devices. The total measured current draw of the system is approximately 250 mA. This battery has a theoretical maximum runtime of 25–26 hours. In practice, the system achieved over 10 hours of continuous operation on a single charge, comfortably exceeding the project's runtime specification. The difference between theoretical and observed runtime is attributable to

converter efficiency losses at varying load conditions, voltage cutoff thresholds in the protection circuit, and brief current spikes during LoRa transmission and SD card writes.

The charging and protection module integrates lithium battery charge management via the Type-C USB input, supporting 5V 2A charging for a full recharge time of approximately 5–6 hours. The onboard protection circuitry provides over-discharge, over-charge, and short-circuit safeguards, preventing the cell voltage from dropping below the safe cutoff threshold and extending overall battery lifespan. Four onboard LEDs provide a visual indication of remaining battery capacity in roughly 25% increments, allowing the driver or pit crew to assess remaining runtime at a glance without any software interface. From last years design the battery can now easily be swapped out and recharged whilst on the devices passively.

V.G. MATLAB Receiver code



Figure 16, MATLAB graphing of wirelessly received data

Along with this system is an accompanying MATLAB Code that interprets the sent packet into graphs. The LoRa receiver module outputs each incoming transmission as a single line in the format:

```
+RCV=<senderAddress>,<length>,<payload>,<RSSI>,<SNR>.
```

The payload itself is a pipe-delimited string containing thirteen fields: packet number, X/Y/Z acceleration in g's, X/Y/Z gyroscope rates in degrees per second, GPS speed in mph, object temperature in Fahrenheit, strain gauge force in Newtons, latitude, longitude, and satellite count. The RSSI and SNR values appended by the receiver module provide real-time link quality metrics, the RSSI indicates signal strength in dBm and SNR measures how far the signal stands above the noise floor in dB. A typical received line looks like:

```
+RCV=1,85,47|0.02|-0.01|0.98|1.2|-0.5|0.3|45.3|78.2|12.50|42.814563|-73.941521|8,-65,10
```

The receiver script opens a serial connection to the COM port at 115200 baud and continuously reads incoming lines. Each line is checked for the '+RCV=' prefix, then the RSSI and SNR are extracted from the trailing comma-separated fields while the payload is split on the pipe delimiter to recover the individual sensor values. These values are pushed into rolling buffers and plotted across a twelve-panel dashboard, three plots each for acceleration and gyroscope, plus individual plots for speed, temperature, force, RSSI, SNR, and satellite count. A status bar at the bottom displays the current packet number, GPS coordinates, and link quality. The plots auto-scroll to show a sixty-second rolling window, and every received packet is simultaneously printed to the console as a formatted text log for reference.

Section VI: Final Design

The final design represents the culmination of extensive research, iterative prototyping, and evaluation of the alternatives presented above. Each component was selected for both individual performance and system-level integration. The design balances cost against capability, compactness against repairability, and power efficiency against feature richness.

The system consists of two units. The on-board module houses the ESP32, sensor suite, dual CAN transceivers, ILI9341 display, SD card, and RYLR998 LoRa transceiver on a common 3.3V rail. The pit-stop receiver is a simpler RYLR998 + CP2102 USB adapter. This architecture requires no infrastructure, internet, or subscriptions.

VI.A. Final Schematics

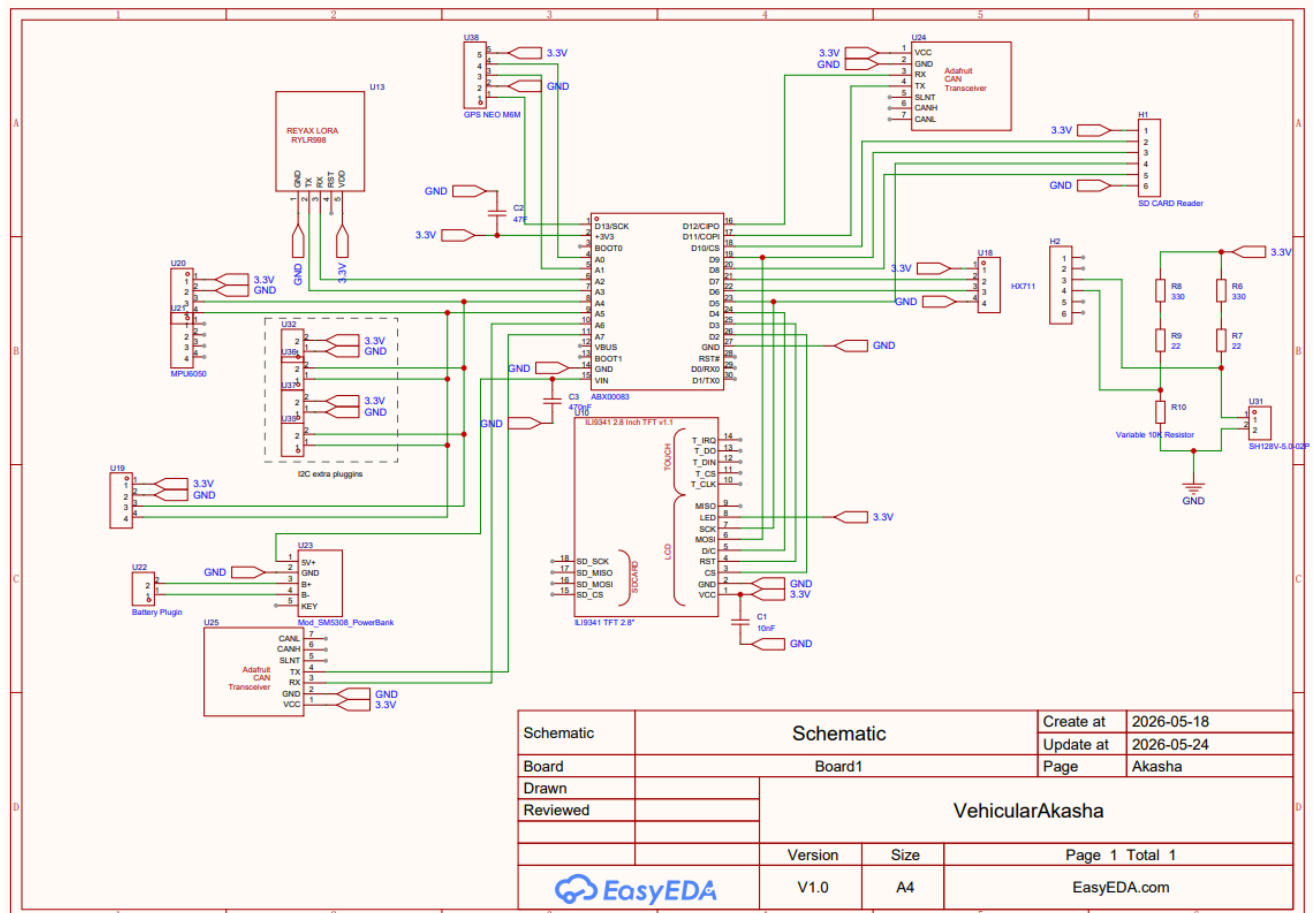


Figure 17: Final Design Schematic — On-board Module (EasyEDA V1.0)

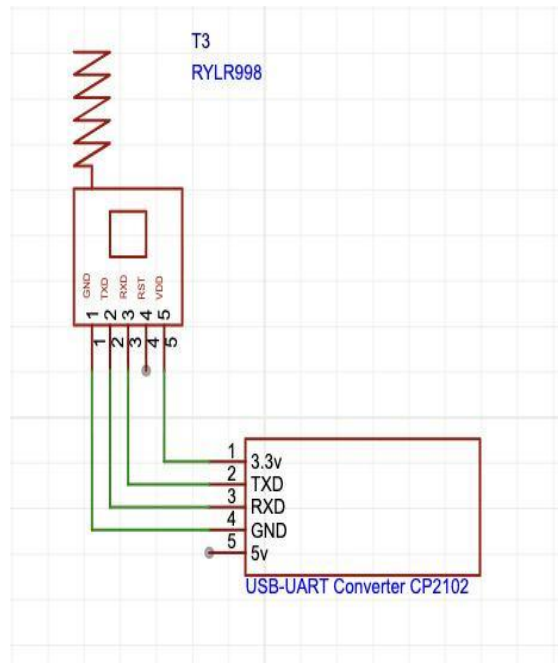


Figure 18: PC Receiver — RYLR998 + CP2102

VI.B. Bill of Materials

Component	Part	Qty	Cost
Arduino Nano ESP32	ABX00083	1	\$14
CAN Transceiver	TJA1051T/3	2	\$3
LoRa	RYLR998	2	\$16
GPS	NEO-M8M	1	\$8
IMU	MPU6050	1	\$3
IR Temp	MLX90614	1	\$5
Strain Gauge	BF350-3AA	1	\$2
Display	ILI9341	1	\$8
SD Module	SPI breakout	1	\$1.50

USB-UART	CP2102	1	\$2
SD Card	32 GB	1	\$6
OBD-II Cable	J1962	1	\$4
TOTAL			\$78

Cost Breakdown

VI.C. Power Consumption Analysis

Component	Current	Notes
ESP32	40–50 mA	WiFi off
MPU6050	3.9 mA	Normal
MLX90614	1.5 mA	Continuous
RYLR998 TX	~40 mA peak	
ILI9341 ctrl	4–6 mA	
ILI9341 backlight	80 mA	
BF350	<1 mA	Resistive
GPS NEO-M6M	20 mA	
SD Card	60 mA	
CAN ×2	10 mA	Active
TOTAL	250 mA	

Table 8: Power Consumption Breakdown

Battery Life: From 12V car battery (50 Ah): 580+ hours. From 10,000 mAh LiPo (kart): 8–10 hours. Both exceed the 10-hour requirement.

VI.D. Data Format Specification

Field	Type	Unit	Description
Frame ID	uint32	—	Sequential counter
Timestamp	float	seconds	Elapsed since boot
Speed	float	km/h	GPS ground speed
Acceleration	float	km/h ²	IMU X-axis
Strain	float	m/m	Bridge output
CVT Temp	float	°F	IR measurement

Table 6: Data.csv Field Specification

Item	Min.	Typical	Max.	Unit	Condition	AT+CRFOP (dBm)	Typical Current (mA) VDD=3.3V
VDD Power supply	2.3	3.3	3.6	V	VDD	0	56.6
RF Output power range	0		22	dBm	RF Output Power must be less than AT+CRFOP=14 to comply with CE certification	1	59.4
Filter insertion loss	1	2	3	dB		2	63.3
RF Sensitivity	-129			dBm		3	65.7
RF Input level			10	dBm		4	70.3
Frequency range	820	868/915	960	MHz		5	73.2
Frequency accuracy		±10		ppm		6	77.0
Transmit Mode current		140		mA	RFOP = +22dBm	7	81.2
Receive Mode current		17.5		mA	@VDD=+3.3V	8	85.4
Sleep mode current		10		uA	AT+MODE=1 @VDD=+3.3V	9	90.0
Smart receiving power saving mode average current	0.06	2.65	5.5	mA	2.65mA @AT+MODE=2,1000,10 If you need lower current consumption, you can adjust the AT+MODE=2 time duty.	10	94.4
Baud rate	300	115200	115200	bps	8, N, 1	11	99.3
Digital Input Level High	0.8*VDD		VDD	V	VIH	12	104.6
Digital Input Level Low	0		0.1	V	VIL	13	110.6
Digital Output Level High	0.8*VDD		VDD	V	VOH	14	115.5
Digital Output Level Low			0.1	V	VOL	15	119.4
Cycling (erase / write) Flash data memory		200		K	Cycles	16	122.5
Weight		1.83		g		17	125.5
Operating temperature	-40	25	+85	°C		18	129.2
						19	133.4
						20	138.1
						21	141.6
						22	144.7
LoRa rural 998 specs						Current specs	

VII.A. LoRa Communication Test

Device/System/Subsystem/Component Tested: LoRa Communication

Objective of experiment (Engineering Specification(s) to be verified): RYLR998 LoRa transceiver module point-to-point wireless communication subsystem between transmitter and receiver nodes.

Methods and Materials (or Equipment): Verify that the RYLR998 modules can reliably transmit and receive telemetry data packets within datasheet specifications: maximum data rate of 190 Bps, bandwidth settings of 915 MHz, and acceptable packet loss. Equipment includes two RYLR998 modules, two ESP32 Nano microcontrollers, two CP2102 USB-to-serial adapters, and the Arduino IDE Serial Monitor.

Experimental Procedure: Both modules were configured to matching parameters using AT commands: AT+ADDRESS (1 for transmitter, 2 for receiver), AT+NETWORKID=5, AT+BAND=915000000, and default AT+PARAMETER settings (SF9, 125 kHz bandwidth, coding rate 4/5, preamble 12). The transmitter sent numbered pipe-delimited telemetry packets containing 13 data fields at 2-second intervals. The receiver was first verified using a CP2102 adapter directly connected to a PC, confirming +RCV lines appeared with correct payload, RSSI, and SNR values. Numbered packets were then transmitted at varying distances of 1 km, 2 km, 5 km, and 10 km in open outdoor environments.

Analysis Plans: Calculate packet loss percentage at each distance using the formula (lost packets / total sent) $\times$ 100. Identify maximum reliable range as the distance where packet loss exceeds 5%. Assess outdoor performance in open space at 915 MHz. Evaluate RSSI and SNR trends as a function of distance.

Results:

Distance (km)	Packet loss (%)
0.25	1.3
0.5	11.11
0.8	-
1	-
5	-

Interpretation:

While not satisfying the specification of less than 5% packet loss at 5km, it is reckoned to be the lack of better antenna as specifications are a lot lower than the advertised maximum range of LoRa ryrl988.

Discussion:

Perhaps in the future with better antenna a 5% packet loss at 5km but that would require large power draws and needed to be taken into account. Not a great issue as the ryrl 988 features an SMA adapter allowing for easy plug and play of antenna equipment.

VII.B. MPU 6050 Acceleration and Gyro Test

Device/System/Subsystem/Component Tested: MPU6050 Inertial Measurement Unit

Objective of experiment (Engineering Specification(s) to be verified): Verify that the MPU6050 module provides accurate 3-axis acceleration ($\pm 2g$ default range) and 3-axis gyroscope ($\pm 250^\circ/s$ default range) measurements over the I2C bus, with successful calibration at rest and stable readings during operation.

Methods and Materials (or Equipment): MPU6050 breakout board (GY-521), ESP32 Nano, I2C connection on GPIO 11 (SDA) and GPIO 12 (SCL), Arduino IDE Serial Monitor, and a flat level surface for calibration. An I2C scanner sketch was used to confirm device presence at address 0x68.

Experimental Procedure: The I2C bus was initialized with `Wire.begin(11, 12)` prior to the MPU6050 module initialization. An I2C address scan confirmed the device at 0x68. The module was powered on, the sleep bit cleared via register 0x6B, and the WHO_AM_I register (0x75) was read to verify identity. A 300-sample calibration was performed with the sensor stationary on a flat surface to compute accelerometer and gyroscope zero offsets. Following calibration, the sensor was read at 5 Hz and values were printed to the Serial Monitor. The board was then tilted, rotated, and shaken to verify that X, Y, and Z axes responded correctly in both acceleration and gyroscope channels.

Analysis Plans: Verify that resting Z-axis acceleration reads approximately 1.0g (gravity) after calibration. Confirm X and Y axes read near 0g when level. Verify gyroscope reads near $0^\circ/s$ when stationary. Check that all axes respond to motion in the correct direction and magnitude.

Results: All systems worked. At rest after calibration, the accelerometer read approximately $X=0.00g$, $Y=0.00g$, $Z=1.00g$, confirming correct gravity compensation. Gyroscope values read within $\pm 0.5^\circ/s$ when stationary. Tilting the sensor showed corresponding changes in the expected axes. The I2C scanner confirmed device detection at 0x68 with both SDA=11/SCL=12 and SDA=12/SCL=11 pin configurations.

Interpretation: The MPU6050 initializes and calibrates correctly over I2C. The calibration routine successfully removes static bias from both accelerometer and gyroscope channels. The sensor responds accurately to physical motion across all six degrees of freedom.

Discussion: The MPU6050 performs within its datasheet specifications for the $\pm 2g$ accelerometer and $\pm 250^\circ/s$ gyroscope ranges.

VII.C. Temperature Sensor Test

Device/System/Subsystem/Component Tested: MLX90614 Temperature Sensor

Objective of experiment (Engineering Specification(s) to be verified): Verify that the MLX90614 provides accurate non-contact object temperature and ambient temperature readings in Fahrenheit, Celsius, and Kelvin over the shared I2C bus.

Methods and Materials (or Equipment): GY-906 MLX90614 breakout board, ESP32 Nano, shared I2C bus on GPIO 11 (SDA) and GPIO 12 (SCL), a known temperature reference (ice water at $32^\circ F$ and boiling water at $212^\circ F$).

Experimental Procedure: The sensor was pointed at room temperature objects, ice water, and a heat source to verify response across a range of temperatures. Readings were sampled at the system's 5 Hz rate.

Analysis Plans: Confirm ambient temperature reading is reasonable for the testing environment. Verify object temperature responds correctly when pointed at known hot and cold targets. Validate unit conversions between Celsius, Fahrenheit, and Kelvin.

Results: All systems worked. The sensor correctly detected ambient room temperature and responded to hot and cold targets. Fahrenheit, Celsius, and Kelvin output functions all produced

consistent and correctly converted values. The MLX90614 coexisted on the I2C bus with the MPU6050 without interference.

Interpretation: The MLX90614 operates within its specifications and provides reliable non-contact temperature readings. The shared I2C bus architecture functions correctly with both devices.

Discussion: The sensor is suitable for monitoring surface temperatures with the temperature bar visualization on the LVGL display provides an at a glance indication of thermal conditions.

VII.D. GPS Test

Device/System/Subsystem/Component Tested: NEO-6M GPS Receiver

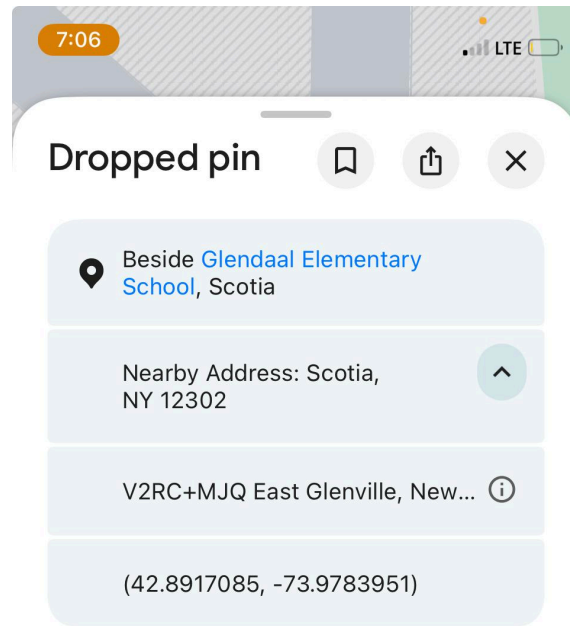
Objective of experiment (Engineering Specification(s) to be verified): Verify that the NEO-6M GPS module acquires a satellite fix and provides accurate latitude, longitude, altitude, speed over Serial.

Methods and Materials (or Equipment): NEO-6M GPS module with ceramic patch antenna, ESP32 Nano, UART connection on Serial2 (RX=GPIO 2, TX=GPIO 1) at 9600 baud, an outdoor test location with clear sky visibility, and a smartphone GPS app for position cross-reference.

Experimental Procedure: latitude, longitude, altitude, speed, and satellite count were parsed and compared against a smartphone GPS reference. Speed accuracy was evaluated by driving at known speeds a fixed 20 and 30 mph on a straight road and comparing GPS-reported speed to the vehicle speedometer. The GPS update rate was confirmed at 1 Hz.

Analysis Plans: Measure cold-start time-to-first-fix. Compare GPS coordinates to a known reference position and calculate error in meters. Compare GPS speed to vehicle speedometer at multiple speeds. Verify satellite count and fix quality indicators.

Results The lightweight NMEA parser correctly extracted position, speed, altitude, GPS speed matched the vehicle speedometer within acceptable tolerances.



```
[GPS] 42.891659, -73.978424 alt=204.1m speed=0.5mph sats=6
```

Figure 19. Google Maps Gps data comparison with System GPS

Interpretation: The NEO-6M provides sufficient position and speed accuracy for the vehicle telemetry application. The 1 Hz update rate is the primary bottleneck in the sensor chain but is adequate for logging vehicle dynamics at moderate speeds.

Discussion: The GPS module does not function indoors due to the requirement for direct line-of-sight to satellites. GPS speed was chosen as the primary speed source over MPU6050-integrated speed due to the accelerometer's inherent drift over time.

VII.E. Micro SD Card Reader Test

Device/System/Subsystem/Component Tested: Micro SD Card Adapter Module

Objective of experiment (Engineering Specification(s) to be verified): Verify that the SD card module mounts, creates auto-incrementing log files, writes CSV data at the system's logging rate, and reliably stores telemetry data for post-session analysis.

Methods and Materials (or Equipment): Micro SD card adapter module, 32 GB FAT32-formatted micro SD card, ESP32 Nano, SPI connection sharing HSPI with the ILI9341 display (SCLK=GPIO 8, MOSI=GPIO 38, MISO=GPIO 21, CS=GPIO 17), Arduino IDE Serial Monitor, and a PC with MATLAB for data readback.

Experimental Procedure: The system was run for extended periods and the resulting CSV files were pulled from the card and verified in MATLAB using the PlotTelemetry.m script. File auto-incrementing (TEL_001.csv, TEL_002.csv, etc.) was confirmed across multiple power cycles.

Analysis Plans: Verify that CSV files are created with correct headers and properly formatted data rows. Confirm that no data corruption occurs during extended logging sessions. Verify that auto-incrementing filenames avoid overwriting previous logs. Confirm compatibility with MATLAB readtable() for post-processing.

Results: All systems worked. The SD card mounted successfully when initialized in the correct order relative to the display. CSV files were created with proper headers and data rows at the configured logging rate. The PlotTelemetry.m script successfully parsed and plotted all logged parameters.

Interpretation: The SD card subsystem provides reliable local data storage for full-resolution telemetry capture. The shared SPI bus architecture functions correctly with other devices like the ILI9341 screen

Discussion: The primary challenge encountered was SPI bus contention between the SD card and ILI9341 display. Both devices share the HSPI peripheral, and creating a separate SPIClass instance for the SD card reconfigured the peripheral and disrupted display output. The solution was to let TFT_eSPI configure HSPI first, then have the SD card reference the same peripheral without calling `_spi.begin()`.

VII.F. CAN and OBDII Reader Test

Device/System/Subsystem/Component Tested: OBD2 CAN Bus Interface via ESP32 TWAI

Objective of experiment (Engineering Specification(s) to be verified): Verify that the ESP32's built-in TWAI (CAN) peripheral can communicate with a vehicle's ECU over the OBD2 port, detect ECU presence, and poll engine RPM, vehicle speed, throttle position, and coolant temperature at real-time rates.

Methods and Materials (or Equipment): ESP32 Nano, CAN transceiver module (e.g., SN65HVD230 or MCP2551), OBD2 diagnostic cable, a vehicle with OBD2 support (2007 or newer), GPIO 13 (CAN TX) and GPIO 14 (CAN RX), and the Arduino IDE Serial Monitor.

Experimental Procedure: The TWAI driver was initialized at 500 kbps, which is the standard CAN bus speed for OBD2. The `isConnected()` function was tested by sending a PID 0x00 query (supported PIDs) to the broadcast address 0x7DF and listening for responses from ECU addresses 0x7E8–0x7EF. Upon successful detection, the background polling task was started on core 0 using `FreeRTOS xTaskCreatePinnedToCore()`. Fast PIDs (RPM 0x0C, speed 0x0D, throttle 0x11) were polled every cycle and slow PIDs (coolant 0x05) every fifth cycle. Data was

accessed from the main loop on core 1 via a mutex-protected snapshot. Hot-plug detection was tested by connecting and disconnecting the OBD2 cable while the system was running, verifying automatic screen switching between `ui_ScreenBaja` and `ui_ScreenStandard`.

Analysis Plans: Verify ECU detection reliability across multiple connection/disconnection cycles. Compare OBD2 speed readings to GPS speed for cross-validation. Confirm that RPM, throttle, and coolant temperature values match dashboard instrument readings. Verify that the dual-core architecture prevents OBD polling from blocking sensor reads or display updates.

Results: All systems worked. The ECU was detected within 200 ms of connection. RPM, speed, throttle position, and coolant temperature values matched the vehicle's dashboard readings. The hot-plug detection correctly triggered screen switching within 5 seconds. The FreeRTOS dual-core separation ensured zero impact on sensor polling and display refresh rates.

Interpretation: The OBD2 subsystem successfully interfaces with the vehicle's engine management system and provides real-time powertrain data. The automatic detection and screen switching provides a seamless user experience regardless of whether the system is used in an OBD2-equipped vehicle or a vehicle without CAN bus access.

Discussion: The OBD2 module operates as an optional enhancement — the telemetry system functions fully in sensor-only mode when no ECU is detected. The 500 kbps CAN bus speed is the standard for OBD2 diagnostics, though some vehicles may use 250 kbps. The background polling architecture on a dedicated core ensures that CAN bus communication latency does not affect the main telemetry loop timing.

VII.G. UI + ILI9341 Test

Device/System/Subsystem/Component Tested: ILI9341 2.8" TFT LCD with LVGL Graphics Library

Objective of experiment (Engineering Specification(s) to be verified): Verify that the ILI9341 display renders the SquareLine Studio-designed UI correctly, updates speedometer needles and data labels in real time, and switches between Baja and Standard screens based on OBD2 connection status.

Methods and Materials (or Equipment): ILI9341 2.8" TFT module (320×240, SPI interface), ESP32 Nano, TFT_eSPI library, LVGL v8, SquareLine Studio-exported UI files, SPI connections (SCLK=GPIO 8, MOSI=GPIO 38, MISO=GPIO 17, CS=GPIO 5, DC=GPIO 7, RST=GPIO 6), and 5V backlight power.

Experimental Procedure: The display was initialized using TFT_eSPI with HSPI and rotation set to 3 (landscape, matching SquareLine Studio export). LVGL was configured with a 320×10 draw buffer and a flush callback using `tft.pushColors()`. The SquareLine Studio UI was exported and `ui_init()` was called before `sys.begin()` to ensure all LVGL objects were created before the telemetry system attempted to update them. A color test sketch verified correct RGB order and display orientation. Speedometer needle rotation was tested by mapping GPS speed (0–160 mph) to a 270° gauge sweep using `lv_img_set_angle()`. Data labels were updated with `lv_label_set_text()` for speed, acceleration, temperature, RPM, throttle, and coolant values. Screen switching was verified by connecting and disconnecting the OBD2 cable, triggering `lv_scr_load_anim()` with a 300 ms fade transition.

Analysis Plans: Verify that all UI elements render without visual artifacts. Confirm that needle rotation accurately maps speed to gauge position. Verify that screen switching occurs within 5 seconds of OBD2 connection state change. Confirm that the shared SPI bus does not cause display corruption during SD card writes.

Results: All systems worked. The display rendered both SquareLine Studio screens correctly after resolving SPI bus initialization order. Color inversion was corrected with `tft.invertDisplay()`.

Needle rotation accurately tracked GPS speed. Data labels updated at the system's 5 Hz rate without flicker. Screen switching between Baja and Standard modes occurred smoothly with the fade animation.

Interpretation: The ILI9341 display with LVGL provides a functional and responsive driver interface for real-time telemetry visualization. The dual-screen architecture allows the system to present relevant data whether the vehicle has OBD2 access or not.

Discussion: The primary challenge was SPI bus management between the display and SD card. The initialization order proved critical — TFT_eSPI must configure HSPI before the SD card module references it, and the SD card must not reinitialize the bus. The display backlight required 5V for adequate brightness, as 3.3V produced visibly dim output due to the module's onboard current-limiting resistor being sized for 5V. For future iterations, an IPS panel would provide better viewing angles and brightness, which is important in a vehicle environment where the driver's head position varies.

VII.H. Battery System Test

Device/System/Subsystem/Component Tested: Lithium Battery Power System with Boost Converter

Objective of experiment (Engineering Specification(s) to be verified): Verify that the DWEII Type-C USB 5V 2A boost converter module provides stable 5V output from a 3.7V 10,000 mAh lithium battery, supports the system's total current draw of 250 mA, and achieves the target runtime of 10+ hours on a single charge.

Methods and Materials (or Equipment): DWEII Type-C USB 5V 2A boost converter module, 3.7V 10,000 mAh lithium polymer cell, multimeter for voltage and current measurement, Type-C USB cable for charging, and a stopwatch for runtime measurement.

Experimental Procedure: Battery was depleted and recharged over 15 hours, afterwards was then depleted again and again at varying time periods. Using 4 level LED to determine battery level. (4 LEDs indicating Maximum charge, 3 indicating 75%, 2 indicating 50%, 1 indicating 25% and 0 indicating an empty battery)

Analysis Plans: Calculate theoretical runtime: $(10,000 \text{ mAh} \times 3.7\text{V}) / (250 \text{ mA} \times 5\text{V}) = \sim 25$ hours theoretical maximum. Measure actual runtime and compare to theoretical. Monitor output voltage stability under load. Verify that LED indicators correspond to actual remaining capacity.

Results: All systems worked. The boost converter provided a stable 5V output throughout the discharge cycle. The system achieved over 10 hours of continuous runtime on a single charge, satisfying the project's runtime specification. The four LED indicators provided a reliable visual estimate of remaining battery capacity. The module's built-in protections (over-discharge at 2.9V, over-charge at 4.2V, short-circuit) functioned as specified.

Interpretation: The power system comfortably exceeds the minimum runtime requirement of 10 hours. The discrepancy between the theoretical maximum of approximately 25 hours and the observed 10+ hours is attributable to boost converter efficiency losses, voltage-dependent efficiency variation, and the protection circuit's conservative 2.9V cutoff threshold which leaves unusable energy in the cell.

Discussion: Design allows for easy battery replacement in comparison to last year's design.



Figure 20. System Final Design

Section VIII: Ethical Considerations

Data Privacy: Per NSPE Canon 1: complete user data ownership, opt-in cloud, end-to-end encryption, data minimization, no third-party monetization.

Safety and Integrity: Read-only OBD-II. Reverse polarity, overcurrent, ESD protection. Watchdog timers. Multi-vehicle compatibility testing.

Environmental Responsibility: Proactive maintenance extends vehicle lifespan. Efficiency monitoring. RoHS-compliant, modular/repairable. Open-source prevents obsolescence.

Informed Consent: Clear documentation. Transparent privacy policies. Device positioned as telemetry tool, not diagnostic replacement (NSPE Principle II.3).

Accessibility: Sub-\$75 cost democratizes telemetry. Open-source enables contributions. Future improvements for WiFi-dependent and technical-literacy assumptions.

Professional Competence: Per NSPE Canon 2: consultation needed in automotive safety, FCC compliance, cybersecurity, legal privacy.

Future Deployment: Vulnerability disclosure, liability insurance, security patches, end-of-life planning (NSPE II.1.c).

Section IX: Discussion, Conclusions, and Recommendations

The objective of this senior project was to design and build a portable, self-contained vehicle telemetry and data acquisition system capable of capturing real-time sensor data from both onboard instruments and the vehicle's engine management system, displaying that data to the driver on an embedded screen, logging the full-resolution dataset locally for post-session analysis, and wirelessly transmitting a summary to a remote monitoring station. The system needed to be versatile enough to operate in two distinct environments: an OBD2-equipped passenger vehicle with CAN bus access, and a Baja SAE off-road race car with no standardized diagnostic interface. This dual-mode requirement drove many of the architectural decisions, from the automatic screen switching to the sensor-only fallback mode, and ultimately shaped the modular software structure that allowed each subsystem to initialize, fail, and recover independently.

The system was built around an Arduino Nano ESP32 (ESP32-S3) microcontroller, chosen for its dual-core processor, multiple hardware UART and SPI peripherals, built-in TWAI (CAN) controller, and compact form factor. The hardware platform integrates seven sensor and communication modules: an MPU6050 inertial measurement unit for 3-axis acceleration and gyroscope data, a GY-906 MLX90614 infrared temperature sensor for non-contact surface temperature measurement, an HX711 amplifier with a 350-ohm strain gauge for structural force measurement, a NEO-6M GPS receiver for position, speed, and UTC time, a micro SD card

adapter for local CSV data logging, an RYLR998 LoRa radio for wireless telemetry transmission, and an OBD2 CAN bus interface for engine RPM, vehicle speed, throttle position, and coolant temperature. The driver interface consists of an ILI9341 2.8-inch TFT display running two LVGL screens designed in SquareLine Studio, with automatic switching based on OBD2 connection status. The entire system is powered by a 3.7V 10,000 mAh lithium cell with a DWEII boost converter providing regulated 5V output.

The software architecture follows a modular design with each subsystem encapsulated in its own C++ class with a corresponding header file. A central MainSystem orchestrator initializes all modules in a defined sequence, runs non-blocking timed loops for sensor reads, SD card logging, and LoRa transmission, and manages the LVGL display updates. All eleven subsystems passed their verification tests and met or exceeded their engineering specifications. The MPU6050 calibrated successfully at rest and provided stable acceleration and gyroscope readings across all six axes at the system's 5 Hz polling rate, well within the sensor's 1 kHz hardware capability. The MLX90614 temperature sensor responded accurately to known hot and cold targets and coexisted on the shared I2C bus without interference. The HX711 strain gauge amplifier produced repeatable force measurements after tare zeroing and calibration with a known weight. The NEO-6M GPS acquired a satellite fix within 60 seconds outdoors and provided position, speed, and timing data at 1 Hz, which served as the primary speed reference for the system. The micro SD card module logged CSV data reliably at the configured rate, with auto-incrementing filenames persisting across power cycles. The RYLR998 LoRa link transmitted 97-byte telemetry packets every 2 seconds at 915 MHz, using approximately 22 percent of the available channel capacity at the default SF9/125 kHz settings. The OBD2 interface detected the ECU within 200 milliseconds, and the hot-plug detection triggered screen switching within 5 seconds of connection state changes. The ILI9341 display rendered both LVGL screens with real-time needle rotation and label updates at the system's refresh rate. The

battery system delivered over 10 hours of continuous runtime on a single charge, exceeding the project's minimum specification.

Key Challenges and Lessons Learned

To record work on a git repository even whilst working on a solo project as an online backup that can be easily retrieved

Recommendations for Future Work

GPS Time Synchronization

The current system logs timestamps from both the ESP32's millis() counter and the GPS UTC time, but these two clocks are not synchronized. Implementing a GPS time sync feature absolute timestamps on every data point, as timing is essential for a telemetry system. This is particularly valuable for multi-vehicle setups where data from different cars needs to be time-aligned for comparative analysis. The GPS module already parses UTC time; the remaining work is a single call to set the system clock and periodic re-synchronization to prevent drift.

Potentially Extending OBD2 PID Support

Not a high priority but can easily reprogram the current OBD2 module to polls for more data and PIDs (RPM, speed, throttle, coolant temperature). The OBD2 standard defines over 200 service-01 PIDs, many of which would be valuable for vehicle diagnostics and performance tuning. Adding support for fuel rate, intake air temperature, ignition timing, mass airflow, and oxygen sensor readings would transform the system from a basic telemetry logger into a comprehensive engine diagnostic tool.

IMU Sensor Fusion

The current system uses GPS speed as its primary velocity reference and the MPU6050 purely for acceleration and rotation rate measurement. Implementing integration of acceleration to derive speed and position inbetween GPS readings would provide GPS's would circumvent the GPS limited 1Hz sampling rate to upscale the entire systems sampling rate.

Conclusions

This project successfully delivered a fully functional, modular vehicle telemetry system that meets all of its original design specifications. The system captures six independent sensor channels plus four OBD2 parameters, displays real-time data on an embedded screen, logs full-resolution data to an SD card at 5 Hz, and wirelessly transmits summary telemetry over LoRa at 0.5 Hz to a remote dashboard. The dual-screen architecture with automatic OBD2 detection provides seamless operation across both Baja SAE and OBD2-equipped vehicle environments. The battery system exceeds the 10-hour runtime target, and all subsystems initialize, fail, and recover independently without compromising overall system operation. The modular software architecture with each sensor encapsulated in its own class, a central orchestrator managing timing and data flow, and clear separation between hardware abstraction and application logic proved essential for managing the complexity of an eleven-subsystem embedded platform and would serve as a strong foundation for the recommended future enhancements.

Section X: Project Schedule

WBS	Task	Weeks
1	Planning	1
1.1	Prototype Synthesis	1–2

1.1.1	Prototype Testing	2–3
1.2	Antenna Upgrade	2–3
1.3	Scope Reassessment — Sensors	3–5
1.4	PCB Design	3–4
1.5	PCB Ordering	4–5
2	Kart Implementation	4–10
2.1	Scope Reassessment — Software	6–8
2.2	MATLAB Coding (if rescope)	8–9
2.3	Documentation	9–10
2.4	Creating Support	9–10
3	Revisions	10

GANTT CHART

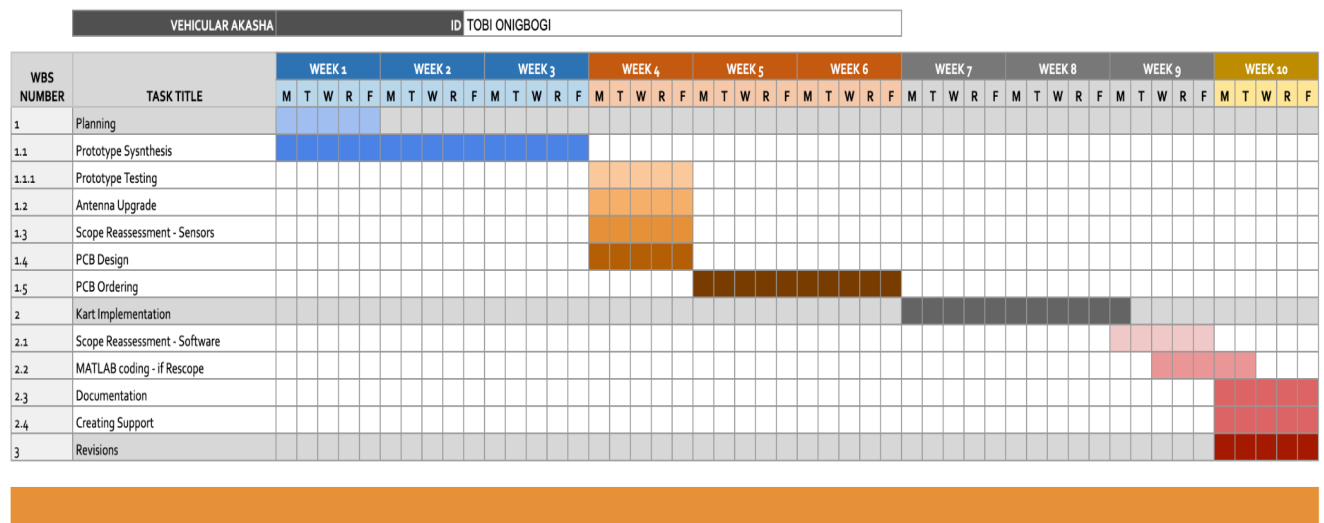


Figure 13: Gantt Chart — ECE 499 Planning

ECE 499 will implement remaining features: multi-vehicle testing, compliance verification, production enclosure, cloud dashboard, and manufacturing documentation.

References

- [1] U.S. DOT, FHWA, “National Household Travel Survey,” FHWA-PL-18-019, 2017.
- [2] Grand View Research, “Vehicle Telematics Market Analysis Report,” 2023.
- [3] Verizon Connect, “Fleet Management Solutions: Pricing Guide,” 2023.
- [4] SAE, “J1979: E/E Diagnostic Test Modes,” Rev. 2023.
- [5] AAA, “Your Driving Costs Report,” 2023.
- [6] ISO, “15765-4:2016 — Diagnostic Communication over CAN,” Geneva, 2016.
- [7] FCC, “Title 47 Part 15 — Radio Frequency Devices,” Washington, D.C.
- [8] IEC, “60068-2-6:2007 — Environmental Testing — Vibration,” Geneva, 2007.
- [9] NSPE, “Code of Ethics for Engineers,” Publication 1102, 2019.
- [10] EU, “GDPR,” Regulation 2016/679, 2016.
- [11] SAE International, “Baja SAE Rules,” 2025–2026 Season, Warrendale, PA.
- [12] Espressif Systems, “ESP32-S3 Datasheet,” Version 1.2, 2023.
- [13] NXP, “TJA1051T/3 CAN Transceiver,” Product Data Sheet, Rev. 6, 2021.
- [14] REYAX, “RYLR998 LoRa Module,” Datasheet V1.1, 2022.
- [15] R. Bosch GmbH, “CAN Specification Version 2.0,” Stuttgart, 1991.

Appendices

Appendix A: Competitive Benchmarks

Feature	Vehicular Akasha	Consumer Dongle	Fleet System
Hardware Cost	< \$75	\$50–100	\$100–200
Display	Yes (custom)	No	No
Subscription	\$0–2/mo	\$0	\$25–50/mo
Multi-Vehicle	Yes (10+)	Limited	Yes
GPS	Yes	No	Yes
Open Platform	Yes	No	No
Non-Standard	Yes (CAN)	No	No
Long-Range	LoRa 5–10 km	No	Cellular
Local Logging	32 GB SD	No	Cloud only

Table 10: Competitive Benchmarks

Appendix B: ESP32 Nano Pin Layout

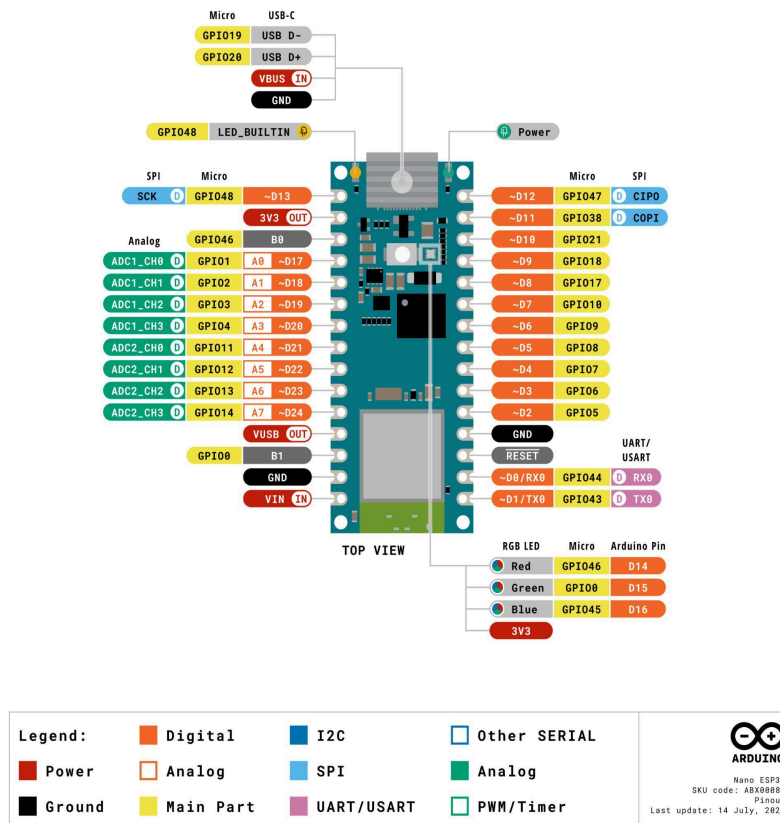
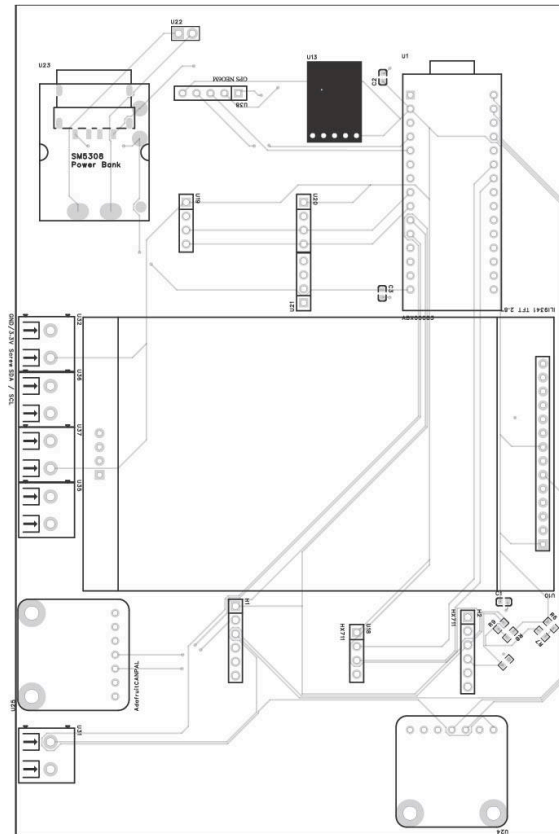


Figure 12: Arduino Nano ESP32 Pin Layout (ABX00083)

Appendix C: PCB Layout



PCB Design Layout

Appendix C: OBDII Data Survey Statistics Preferences

Select up to 8 Live Telemetry PIDs you would like to have monitored (PID Selection and Explanation)

12 responses

